

PROGRAMACIÓN REACTIVA Y SISTEMAS REACTIVOS

J. S. Morales López

7690-08-6790 Universidad Mariano Gálvez

Seminario de tecnologías de información

jmorales115@miumg.edu.gt

Repositorio en GitHub - Sofia Morales

Resumen

El presente artículo analiza los principios y las buenas prácticas necesarias para desarrollar aplicaciones reactivas capaces de responder oportunamente ante eventos, cambios de carga y fallos. A partir de la revisión de fuentes especializadas, se diferencia la programación reactiva, empleada para administrar flujos de datos asíncronos y no bloqueantes, de los sistemas reactivos, entendidos como un enfoque arquitectónico completo. Entre los principales resultados se identificaron cuatro propiedades: capacidad de respuesta, resiliencia, elasticidad y orientación a mensajes. También se reconocieron prácticas como documentar el flujo de eventos, aplicar contrapresión, reducir el estado compartido, aislar los fallos, incorporar métricas y ejecutar pruebas de carga y recuperación. Estas medidas favorecen el uso eficiente de los recursos y la continuidad del servicio; sin embargo, requieren una planificación cuidadosa, porque la asincronía puede dificultar la depuración y el monitoreo. Se concluye que una aplicación no se vuelve reactiva solamente por utilizar tareas asíncronas, sino por integrar principios de diseño, control del flujo, observabilidad y recuperación ante fallos.

Palabras clave: programación reactiva, sistemas reactivos, resiliencia, elasticidad, mensajería asíncrona.

Desarrollo del tema

La programación reactiva y los sistemas reactivos responden a necesidades relacionadas, pero no representan exactamente el mismo concepto. La programación reactiva se aplica principalmente al procesamiento de flujos de datos y eventos asíncronos, mientras que los sistemas reactivos incorporan estos principios dentro de un enfoque arquitectónico más amplio (Bonér, 2023).

Para implementar este enfoque adecuadamente, las buenas prácticas deben considerarse desde la etapa de diseño. El flujo de eventos debe mostrar el origen de los datos, los componentes encargados de procesarlos y las respuestas generadas. Esto facilita la identificación de errores, retrasos y posibles puntos de saturación. Gillis y Nolle (2024) también recomiendan mantener actualizado el diagrama del flujo y revisar el acceso a las bases de datos para evitar latencia innecesaria.

Una práctica especialmente importante es la contrapresión. Este mecanismo permite que el consumidor indique cuánto puede procesar y evita que un productor envíe más información de la que el sistema puede atender. De esta manera, se reduce el riesgo de utilizar recursos sin límite y de acumular eventos pendientes durante periodos de alta demanda (Bonér, 2023).

También es necesario aislar los componentes para impedir que el fallo de uno afecte al sistema com-

pleto. El monitoreo mediante registros, métricas y alertas permite detectar comportamientos anormales y conocer el estado de la aplicación. Finalmente, las pruebas de carga, concurrencia y recuperación ayudan a comprobar si el sistema conserva su capacidad de respuesta ante fallos o variaciones en la cantidad de solicitudes.

- Diseñar y documentar el flujo de eventos.
- Emplear operaciones asíncronas y no bloqueantes.
- Aplicar contrapresión para evitar que un productor sature al consumidor.
- Reducir el estado compartido entre componentes.
- Aislar los fallos y establecer mecanismos de recuperación.
- Implementar registros, métricas y monitoreo.
- Realizar pruebas de carga, concurrencia y fallos.
- Evaluar si el enfoque reactivo realmente es necesario para la aplicación.

TechTarget recomienda diagramar los flujos de eventos, reducir componentes con estado y revisar el acceso a las bases de datos para evitar latencia (Gillis & Nolle, 2024). Akka también explica que la programación reactiva es una técnica interna, mientras que un sistema reactivo representa un enfoque arquitectónico más amplio (Bonér, 2023).

Programación reactiva

La programación reactiva permite gestionar flujos de información y responder a los cambios que se producen en los datos. Se apoya en operaciones asíncronas y no bloqueantes para que la aplicación pueda continuar atendiendo otros eventos mientras espera que finalice una operación, como una consulta a una base de datos o una solicitud a un servicio externo.

Según Pérez (2026), la programación reactiva permite que los componentes respondan a modificaciones en los datos y procesen eventos mediante flujos asíncronos sin detener la ejecución de la aplicación. Sus principales principios son:

- Flujos de datos observables.
- Propagación de cambios.
- Manejo eficiente de eventos.

Sistemas reactivos

Los sistemas reactivos, son más flexibles, con bajo acoplamiento y escalables. Son significativamente tolerantes a fallos y por ende cuando fallan responden de manera oportuna. Estos sistemas presentan cuatro características fundamentales según (Bonér et al., 2014):

- **Responsivos:** mantienen tiempos de respuesta oportunos y consistentes para proporcionar un servicio de calidad.
- **Resilientes:** conservan su capacidad de respuesta ante fallos mediante el aislamiento, la contención y la recuperación de los componentes afectados.
- **Elásticos:** aumentan o reducen los recursos utilizados según los cambios en la carga de trabajo.
- **Orientados a mensajes:** utilizan mensajes asíncronos para reducir el acoplamiento entre componentes y controlar el flujo de información.

Observaciones y comentarios

El enfoque reactivo puede ser útil cuando una aplicación debe procesar eventos constantes, responder en tiempo real o adaptarse a cambios de carga. Sin embargo, su implementación también incrementa la complejidad del desarrollo, especialmente durante la depuración y el seguimiento de operaciones asíncronas. Por esta razón, no debería utilizarse únicamente porque sea una tecnología moderna. Antes de adoptarlo, es necesario analizar las necesidades, el volumen de eventos y los recursos disponibles.

Conclusiones

1. La programación reactiva facilita el procesamiento de eventos y flujos de datos asíncronos sin bloquear innecesariamente la ejecución de la aplicación.
2. La contrapresión permite controlar la cantidad de información procesada y evita que un productor sature la capacidad del consumidor.
3. La resiliencia, la elasticidad y el aislamiento de fallos permiten conservar la continuidad del servicio ante errores o cambios en la demanda.
4. El enfoque reactivo puede utilizarse en aplicaciones de IoT, sistemas distribuidos y servicios en tiempo real; sin embargo, su implementación debe evaluarse según las necesidades y la complejidad del proyecto.

Bibliografía

- Bonér, J. (2023, 24 de octubre). *Reactive programming vs. reactive systems*. Akka. <https://akka.io/blog/reactive-programming-versus-reactive-systems>
- Bonér, J., Farley, D., Kuhn, R., & Thompson, M. (2014, 16 de septiembre). *El Manifiesto de Sistemas Reactivos*. <https://www.reactivemanifesto.org/es>
- Gillis, A. S., & Nolle, T. (2024, 14 de junio). *What is reactive programming?* TechTarget. <https://www.techtarget.com/searchapparchitecture/definition/reactive-programming>
- Pérez, A. (2026, 11 de mayo). *Cómo aplicar la programación reactiva en aplicaciones web modernas*. OBS Business School. <https://www.obsbusiness.school/blog/como-aplicar-la-programacion-reactiva-en-aplicaciones-web-modernas-cp>